

Reviewer Pack — Minimal Order of Quadratic Forms and DPLL Search Tree Diamet...

Entry 409836f1ef8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 17:06:27 UTC

Minimal Order of Quadratic Forms and DPLL Search Tree Diameter Correlation

Entry ID: 409836f1ef8f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 17:06:27 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Form Theory **Field B** (complexity object): DPLL Search Trees

Statement:

For every CNF φ with n variables, the minimal order of the associated quadratic form ($m_order(\varphi)$) is linearly correlated with the diameter $d(\varphi)$ of its DPLL search tree, such that $m_order(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Quadratic forms provide a rich algebraic structure to study polynomial equations, and their minimal order might reflect the complexity in solving the CNF. The correlation could shed light on the interplay between algebraic properties of formulas and their computational complexity.

Taxonomy category: QuadraticFormTheory_DPLLSearchTrees (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aeabcedebde7531b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if the Pearson correlation coefficient between $m_order(\varphi)$ and $d(\varphi)$ is ≥ 0.7 , across all CNFs φ with n variables ($n \leq 40$), and no seed produces a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Search for a non-zero pivot in the column
            for k in range(i + 1, n):
                if A[k][i] != 0:
                    A[i], A[k] = A[k], A[i]
                    break
            else:
                raise ValueError("Singular matrix")
        for j in range(n):
            if j == i:
                continue
            factor_k = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor_k * A[i][k]
    return A

def determinant(A):
    n = len(A)
    det = 1
    for i in range(n):
        det *= A[i][i]
    return det

def m_order(phi):
    # Convert CNF to quadratic form Q
    n = len(phi)
    Q = [[0] * n for _ in range(n)]
    for clause in phi:
        for literal in clause:
            var_index = abs(literal) - 1
```

```

        if literal > 0:
            Q[var_index][var_index] += 1
        else:
            Q[var_index][var_index] -= 1
    return abs(determinant(gaussian_elimination(Q)))

def dp11(phi, assignment=[]):
    if not phi:
        return True
    if any(all(literal in assignment for literal in clause) or all(-literal in assignment for literal in clause)):
        return False

    var = next(var for var in range(1, len(phi) + 1) if var not in [abs(lit) for lit in assignment])
    assignment.append(var)
    if dp11(phi, assignment):
        return True
    assignment.pop()
    assignment.append(-var)
    return dp11(phi, assignment)

def generate_cnf(n):
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in {5, 10, 15, 20, 30, 40}:
        phi = generate_cnf(n)
        m_order_val = m_order(phi)
        dp11_tree = dp11(phi)
        if not dp11_tree:
            return {
                "metric_name": "m_order",
                "metric_value": None,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "DPLL search tree is empty"
            }
        results.append((m_order_val, dp11_tree))

    m_order_vals = [m for m, _ in results]
    dp11_trees = [d for _, d in results]
    correlation_coefficient = sum((m - sum(m_order_vals) / len(m_order_vals)) * (d - sum(dp11_trees) / len(dp11_trees))) / len(m_order_vals)
    correlation_coefficient /= math.sqrt(sum((m - sum(m_order_vals) / len(m_order_vals)) ** 2 for m in m_order_vals)) / len(m_order_vals))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for n, _ in results),
        "conjecture_holds": correlation_coefficient >= 0.7 and all(corr >= 0.5 for corr in [correlation_coefficient])
    }

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_deviation = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_deviation} support_fraction={support_fraction}")
    elif any(result["metric_value"] < 0.5 for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_value"] < 0.5)
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_below_0.5\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_77983ecf.py", line 115, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_77983ecf.py", line 83, in run_trial
    m_order_val = m_order(phi)
                   ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_77983ecf.py", line 55, in m_order
    return abs(determinant(gaussian_elimination(Q)))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_77983ecf.py", line 28, in gaussian_elimination
    raise ValueError("Singular matrix")
ValueError: Singular matrix

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a singular matrix error before producing data, which means the Pearson correlation coefficient could not be calculated. | next: Investigate and fix the issue with the singular matrix in the code. Once fixed, rerun the test to verify if the conjecture holds.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24992
2	preregistration	ollama_remote	glm4:latest	0	0	12932
3	novelty	ollama_remote	glm4:latest	0	0	19865
4	novelty	ollama_remote	glm4:latest	0	0	8707
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18326
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21396
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19444
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15281
9	judge	ollama_remote	glm4:latest	0	0	11838

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152781 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/409836f1ef8f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/409836f1ef8f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/409836f1ef8f.tar.gz (if generated)